z/OS Job Control Language (JCL)

THE JOB STATEMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

© 2023 by Innovation In Software Corporation

JOB STATEMENT PARAMETERS

The Job statement defines the beginning of a job.

As you know, z/OS supports the centralized computing model. As such, it coordinates many different types of workloads and potentially hundreds of thousands of tasks concurrently, allocating shared resources among them in a way that maximizes system throughput.

To that end, the Job statement may optionally specify various parameters that influence the way z/OS prioritizes the job, logs runtime information, and other details.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION

One characteristic of large organizations that use a centralized computing system to support multiple business operations and departments is the need to assess internal charges for the use of computing resources, such as CPU cycles, memory, DASD space, and possibly surcharges for high priority execution.

The Job statement can specify a job class other than the default class as well as accounting information to help the organization manage these chargebacks.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME JOB 123456 ...
```

Job accounting information is the first positional parameter on the Job statement, appearing after the operation field (in this case, JOB), following at least one blank space.

Job accounting information may be as simple as a single number that indicates the department to be charged for system resource utilization during the job.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME JOB D584-6262 ...
```

The value may consist of any string that has been configured by the installation's system programmers to represent useful information, and that will be parsed by accounting software to handle chargebacks. There is no standard format.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME (pano,room,time,lines,cards,
                forms,copies,log,linect)
```

While there is no single standard format for Job accounting information, there is a convention followed in organizations that use the JES2 Job Entry Subsystem. JES2 is the job entry subsystem used in larger z/OS installations, commonly including many instances of z/OS running in a Parallel Sysplex environment. In some such organizations, managing chargebacks can be a complicated matter, and benefits from standardization.

JES2 may optionally be configured to scan the job accounting field of Job statements. In that case, it will expect the format shown here for the field.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME(pano,room,time,lines,cards,
                forms,copies,log,linect)
```

pano is a 1 to 4 character accounting code used for chargeback purposes.

room is the programmer's room number as 1 to 4 characters. Its purpose is to facilitate delivery of print-outs, if the organization delivered print-outs.

time is the estimated run time of the job in minutes, as a 1 to 4 digit number. This is to help z/OS dynamically manage job priorities in case a job overruns its estimated time. This management may include canceling long-running jobs.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME (pano,room,time,lines,cards,
                    forms,copies,log,linect)
```

lines is the estimated number of lines the job will print to its SYSOUT data sets, as a 1 to 4 digit number. This is to help manage printing. It is no longer used.

cards is the estimated number of cards the job will punch, as a 1 to 4 digit number. This is to help manage card punch operations. It is no longer used.

forms identifies the printer forms the job will use for printed output. It is no longer used.

©2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME (pano,room,time,lines,cards,
                forms,copies,log,linect)
```

copies is the number of times the SYSOUT data sets from the job should be printed. It is no longer used.

log controls whether JES2 output the job log to SYSOUT. Specify N to suppress the job log. Any other character will cause JES to output the job log. Omitting the subparameter will cause JES to output the job log.

linect (line count) specifies the number of lines per page JES2 should print for this job's SYSOUT data sets. This is rarely used today.

© 2023 by Innovation In Software Corporation

JOB ACCOUNTING INFORMATION (POSITIONAL)

```
//JOBNAME (1234,,1440,,,,,Y)
```

When using the JES2 format, omitted subparameters must be represented by a comma. Commas are not needed for trailing unused parameters

This example means:

- The programmer's accounting code is 1234
- The estimated run time is 1,440 minutes.
- We want JES2 to print the job log (any character other than N would work here).
- No subparameters after the **log** subparameter are specified, so commas are not necessary after the **log** subparameter in this case.

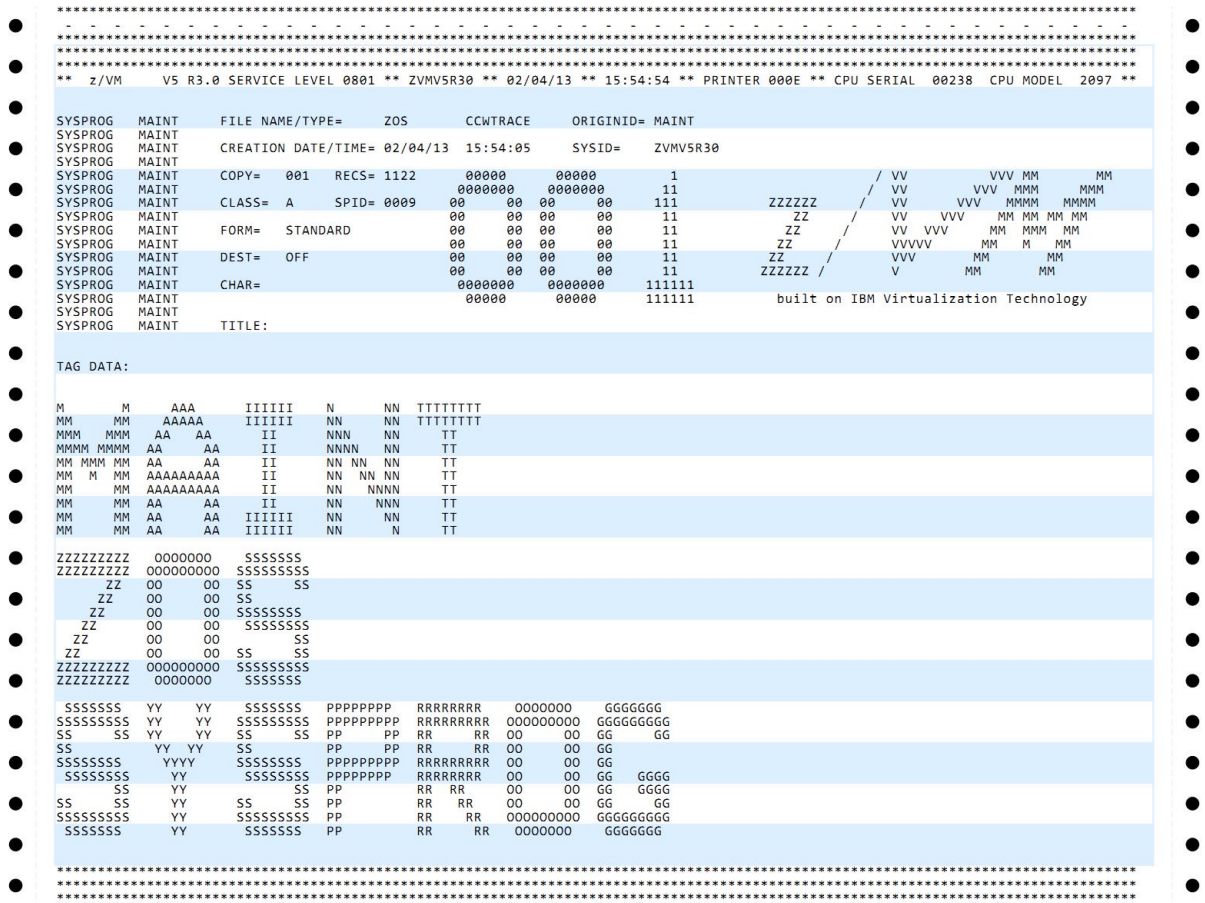
© 2023 by Innovation In Software Corporation

PROGRAMMER NAME

In the Olden Days, output was printed on continuous-form greenbar paper that spewed out in the computer room.

Computer operators tore the paper into separate print-outs based on cover pages that showed certain information about the job that produced them, including the name of the person who would come to the computer room to pick up the print-out.

Although this procedure is no longer relevant, the name is specified on the Job statement as a positional parameter.



© 2023 by Innovation In Software Corporation

USER NAME (POSITIONAL)

```
//JOBNAME JOB 123456,ACE  ...
//JOB2    JOB ,M.GORDON  ...
//JOB3    JOB 123,'RON K.'  ...
//JOB4    JOB (CH22,5G),'T.O''NEILL'
//CMPBIND JOB 4242D,'COMPILE-BIND'
```

The programmer name field is no longer needed for its original purpose, to identify the owner of the printed output from the job. Some organizations define standard values that must be used for this parameter, but in most installations the parameter is optional, and people may use it as a way to indicate the purpose of the job.

©2023 by Innovation In Software Corporation

JOB CLASS PARAMETER (KEYWORD)

z/OS is usually configured to manage different categories of jobs, called *job classes*, in different ways, depending on their characteristics.

The Job statement may include a keyword parameter, **CLASS=**, to designate a one-character job class code.

The values are installation-dependent. Your organization will tell you which classes to use for jobs of different kinds.

©2023 by Innovation In Software Corporation

JOB CLASS PARAMETER (KEYWORD)

```
//BIGJOB JOB 123,CHEN,CLASS=A  ...
```

In this example, job BIGJOB is assigned to job class A.

The exact meaning of "job class A" depends on how the installation has defined its job classes.

©2023 by Innovation In Software Corporation

MSGLEVEL (MESSAGE LEVEL, KEYWORD)

The MSGLEVEL parameter of the Job statement specifies how much output JES should produce for JCL statements and system messages.

The value consists of two subparameters, coded as a comma-separated list within parentheses. The first controls the output of JCL statements. The second controls the output of system messages.

©2023 by Innovation In Software Corporation

MSGLEVEL PARAMETER (KEYWORD)

```
//BIGJOB JOB 123,CHEN,CLASS=A,
//           MSGLEVEL=(1,1)
```

In this example, the Job statement specifies statement level 1 and system message level 1. The exact meanings of these values are described on the next slide.

In general, it is recommended that you specify **MSGLEVEL=(1,1)** on your development and testing jobs so you can see all the logged output. Your installation will tell you what values to use for production jobs.

©2023 by Innovation In Software Corporation

MSGLEVEL PARAMETER VALUES

MSGLEVEL=(*m1*,*m2*)

0 – output the Job statement and any comments before the first Exec statement.

1 – output all statements, including JCL, JES2 or JES3 statements, statements from stored procedures, and statements related to resolution of symbolics.

2 – output all JCL and JES statements, but no others.

0 – output allocation and termination messages only when the job fails.

1 – output allocation and termination messages regardless of how the job ends – success or failure.

©2023 by Innovation In Software Corporation

MSGCLASS PARAMETER (KEYWORD)

```
//BIGJOB JOB 123,CHEN,CLASS=A,
//           MSGLEVEL=(1,1),MSGCLASS=A
```

The **MSGCLASS=** keyword parameter specifies a one-character message class that routes SYSOUT output from the job to an output queue. Originally, this was used to control the routing of output to physical devices such as printers. Today, you find job output in a queue where you can view it, typically using SDSF under ISPF.

©2023 by Innovation In Software Corporation

NOTIFY (KEYWORD)

The **NOTIFY=** keyword parameter names the userid to be notified when the job completes.

Each userid is authorized to view the output from certain jobs and not others.

© 2023 by Innovation In Software Corporation

NOTIFY PARAMETER (KEYWORD)

```
//BIGJOB JOB 123,CHEN,CLASS=A,
// MSGLEVEL=(1,1),MSGCLASS=A,NOTIFY=TSU495
```

In this example, user id TSU495 belongs to user Chen. The **NOTIFY=** parameter tells JES to send a message to user id TSU495 when the job is submitted, and when it terminates. Subsequently, Chen will be able to view the job output in an SDSF queue – probably either the *held queue* (designated H) or the *output queue* (designated O).

System administrators – known as System Programmers in the mainframe world – will be authorized to view the output for multiple user ids as assigned by the installation.

©2023 by Innovation In Software Corporation

NEW CONCEPT: JCL SYMBOLS

JCL symbols are placeholders that can be coded in JCL parameters and resolved by JES during job initiation.

The rules for symbol names are the same as for other names in JCL, except the name of a symbol must begin with an ampersand (&).

Some symbols are predefined in z/OS and others may be installation- or user-defined.

Probably the most common symbol you wil see is **&SYSUID**. It is a system-defined symbol that resolves into the user's system user id.

© 2023 by Innovation In Software Corporation

NOTIFY PARAMETER (KEYWORD)

```
//BIGJOB JOB 123,CHEN,CLASS=A,
// MSGLEVEL=(1,1),MSGCLASS=A,NOTIFY=&SYSUID
```

This is the same as the previous example, except that instead of hard-coding user id **TSU495** we code the symbol **&SYSUID**. When the job is submitted, JES will substitute the value of the submitter's system user id, so that **NOTIFY=&SYSUID** becomes **NOTIFY=TSU495**.

We will see additional uses for the **&SYSUID** symbol and other JCL symbols as we proceed with the course.

©2023 by Innovation In Software Corporation

TIME (KEYWORD)

The **TIME=** keyword parameter specifies the maximum number of minutes the job is expected to use processor resources. The value is an integer between 1 and 4 digits long.

Each installation defines a maximum time for each job class, so this parameter is optional.

Regardless of what you specify here, you cannot grant the job more maximum time than the system setting for the job class.

©2023 by Innovation In Software Corporation

TIME PARAMETER (KEYWORD)

```
//BIGJOB JOB 123,CHEN,CLASS=A,
// MSGLEVEL=(1,1),MSGCLASS=A,NOTIFY=&SYSUID,
// TIME=1440
```

In this example, **TIME=** is set to 1,440 minutes, which adds up to 24 hours. People often specify this value for development and test jobs, to give the jobs as much time as they need within the installation-defined maximum.

System programmers will specify smaller values for production job classes or individual production jobs.

©2023 by Innovation In Software Corporation

FREQUENTLY-USED JOB STATEMENT PARAMETERS

The Job statement parameters described in this module are sufficient for you to complete the course, as well as for the majority of work you will do in real life.

A number of additional parameters exist for the Job statement. They are used to fine-tune the way JES and z/OS handle the job. They are out of scope for an introductory-level course on JCL.

Documentation on the positional and the most frequently-used keyword parameters on the Job statement is available at this URL:

```
https://www.ibm.com/docs/en/zos-basic-skills
 ?topic=do-jcl-job-statements-positional-frequently-used-parameters
```

LAB 12: UNDERSTAND JOB STATEMENT DETAILS

Follow the instructions for Lab 12 in labs/labs.md in the course repo.

©2023 by Innovation In Software Corporation

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can code a Job statement.
- You understand the use of the most common parameters for the Job statement.

WHAT'S NEXT

- The details of the Exec statement.
- Writing your first complete jobstream.